Introducción a React

Vite, JSX y primera aplicación

A01 · Registro de participantes

¿Dónde seleccionábamos los elementos?
¿Qué cambiaba después de registrar?
¿Qué había que actualizar al eliminar?

Localiza selección, evento y cambio del DOM.

DOM · Instrucciones paso a paso

HTML previo: <ul id="lista">
Prueba el fragmento con el documento cargado.

```
const lista = document.querySelector("#lista");
const fila = document.createElement("li");
fila.textContent = "Ana · 2.º DAW";
lista.append(fila);
fila.textContent = "Ana · React";
```

React · Una biblioteca para interfaces

Dividimos la interfaz en componentes.
Describimos qué debe aparecer con nuestros datos.
React gestiona el renderizado y los cambios del DOM.

HTML, CSS y JavaScript siguen siendo la base.

Imperativo y declarativo

Imperativo: indicar cada cambio en el DOM.
Declarativo: describir la interfaz resultante.

```
// JavaScript: ordenamos una modificación
titulo.textContent = `Hola, ${nombre}`;

// JSX dentro de un componente
return <h1>Hola, {nombre}</h1>;
```

React, Node.js, npm y Vite

React	Construcción de la interfaz
Node.js	Ejecuta herramientas fuera del navegador
npm	Gestiona paquetes y ejecuta scripts
Vite	Servidor de desarrollo y preparación del código

Comprobación del entorno

Terminal integrada de VS Code
Cada comando debe mostrar una versión.

```
node --version
npm --version
```

Mi primer proyecto en React

- Lo primero que debemos instalar es entorno de ejecución para JS, en nuestro caso NodeJS.
- Ahora pasaremos a elegir la herramienta de NodeJS que usaremos para crear nuestro proyectos, en nuestro caso usaremos Vite.



Creación de la aplicación

Ejecuta la primera orden en tu carpeta de trabajo.

```
npm create vite@latest primera-app-react
cd primera-app-react
npm install
npm run dev
```

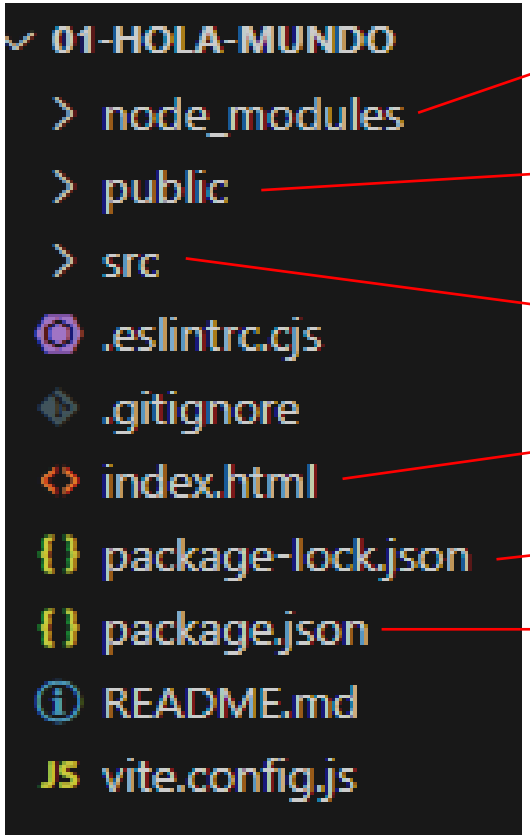
Primera ejecución

Abre la dirección Local que muestra Vite.
Mantén abierta la terminal del servidor.
Guarda un cambio y observa el navegador.

Ctrl + C detiene el servidor.

Mi primer proyecto en React

- Analicemos la estructura de directorio de nuestro proyecto



Es donde se instalan todas las dependencias requeridas por nuestro proyecto.

El directorio público sirve como espacio de almacenamiento para activos estáticos que no serán procesados por Vite

Es donde pasaremos la mayor parte de nuestro tiempo trabajando en nuestro proyecto React. Contiene varios archivos y directorios importantes que forman la base de nuestra aplicación.

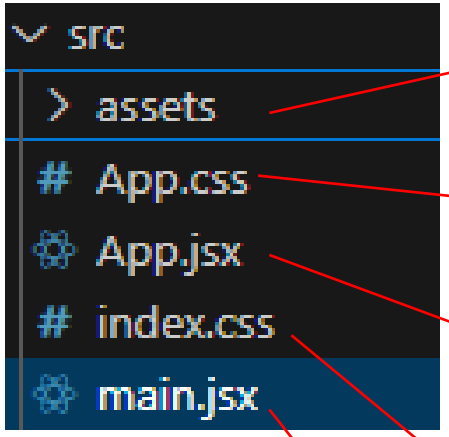
Este archivo tiene un papel especial como el archivo HTML inicial que sirve a nuestra aplicación de una sola página (SPA)

Especifica las versiones exactas que deben instalarse

Contiene detalles clave como el nombre del proyecto, la autoría, el número de versión y los scripts para ejecutar varias tareas. Además, enumera las dependencias, que son las bibliotecas y herramientas externas en las que se basa nuestro proyecto para funcionar correctamente.

Mi primer proyecto en React

- Ahora veremos que hay dentro de la carpeta src, donde estaremos trabajando



Carpeta que contiene los elementos estáticos de nuestra aplicación

La hoja de estilo asociada con el componente de la aplicación, proporciona un estilo específico para este componente.

Este es el componente principal de nuestra aplicación React, que sirve como punto de entrada para nuestra UI

Esta es una hoja de estilo global que aplica estilos a toda la aplicación, proporcionando una apariencia consistente en diferentes componentes.

Este archivo es a menudo el punto de entrada para nuestro código JavaScript, donde importamos y renderizamos el componente principal de la aplicación.

index.html y el elemento root

root es el contenedor de nuestra aplicación.

```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

main.jsx · Punto de entrada

createRoot conecta React con el contenedor.

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode><App /></StrictMode>
);
```

App.jsx · Un componente funcional

Sustituye el contenido inicial de App.jsx.

```
export default function App() {
  return (
    <main>
      <h1>Mi primera aplicación</h1>
      <p>Estamos aprendiendo React.</p>
    </main>
  );
}
```

JSX · JavaScript en las llaves

Las llaves contienen expresiones que producen valores.

```
const nombre = "Ana";
const horas = 6;

return (
  <p>{nombre}: {horas * 2} horas en dos semanas</p>
);
```

Reglas básicas de JSX

Un elemento raíz o un fragmento <>...</>.
Todas las etiquetas deben cerrarse.
className para las clases CSS.
Los componentes empiezan en mayúscula.

```
return (
  <>
    <h1 className="titulo">Hola</h1>
    <input aria-label="Nombre" />
  </>
);
```

className y hoja de estilos

App.jsx importa los estilos; CSS define la apariencia.

```
import "./App.css";

// Dentro del return de App
<h1 className="titulo">Ficha de estudiante</h1>

/* App.css */
.titulo { color: #34796f; }
```

style · Un objeto de JavaScript

Llaves exteriores: expresión JSX.
Llaves interiores: objeto con propiedades CSS.

```
<p style={{ color: "#34796f", fontSize: 24 }}>
  Aprendiendo React
</p>
```

Condicional con operador ternario

condición ? valorSiVerdadero : valorSiFalso

```
const matriculado = true;

return (
  <p>
    {matriculado ? "Matriculado" : "Pendiente"}
  </p>
);
```

Guiado · Corrección de JSX

Localiza y corrige los problemas del fragmento.

```
return (
  <h1 class="titulo">Ficha</h1>
  <p>{if (matriculado) "Alta"}</p>
  <input>
);
```

Reto · Ficha de estudiante

Crea una ficha con nombre, curso y horas semanales.
Muestra «Matriculado» o «Pendiente» con un ternario.
Aplica className y un estilo en línea.
Cambia los datos y comprueba el resultado.

Entrega App.jsx, App.css y dos pruebas anotadas.